

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – OPEN DATA ANALYSIS PROJECT

Course Code:	DSA05	Course Title:	Query Processing for Data Science in Real Time Applications
CO Assessed:	CO4	Assessment:	Tool 3 – Open Data Analysis Project
Weightage:		Total Marks:	50 Marks
CO4 Statement: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.. (BL4, BL5 and BL6)			
Student Name:	S SUVAN SENTHIL	Reg. No.:	192324175
Section / Batch:	2023	Date of Submission:	21-08-2026
Faculty In-charge:	K.Veena devi	Assessment Mode:	Individual Submission

Code of Conduct

I S SUVAN SENTHIL(Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Suvan Senthil

Signature:

Data Collection & Understanding (10 Marks)	Data Cleaning & Preparation (10 Marks)	Exploratory Data Analysis (10 Marks)	Visualization & Dashboard Design (10 Marks)	Demonstration (10 Marks)	Total (50 Marks)

2. Air Quality and Environmental Impact Assessment

Obtain air quality datasets from [World Air Quality Index Project](#) or government open-data portals and analyze pollution levels across cities or regions. Perform exploratory data analysis to identify seasonal variations, pollutant concentration patterns, and correlations between pollutants and meteorological variables. Detect anomalies, visualize geographic distributions using maps, and propose data-driven recommendations for environmental management and public health awareness.

CODE:

```
# =====  
# AIR QUALITY AND ENVIRONMENTAL IMPACT ASSESSMENT  
# Google Colab - Complete Single-Cell Project  
# =====  
  
# -----  
# 1. INSTALL & IMPORT LIBRARIES  
# -----  
!pip -q install folium seaborn  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import folium  
from google.colab import files  
from IPython.display import display  
  
print("=" * 70)  
print("AIR QUALITY AND ENVIRONMENTAL IMPACT ASSESSMENT")  
print("=" * 70)  
  
# -----  
# 2. UPLOAD DATASET  
# -----  
print("\nUpload your CPCB Air Quality CSV file:")  
uploaded = files.upload()  
  
filename = list(uploaded.keys())[0]  
df = pd.read_csv(filename)  
  
print("\nDataset loaded successfully!")  
print("Dataset shape:", df.shape)  
  
# -----  
# 3. INITIAL DATA EXPLORATION  
# -----  
print("\n" + "=" * 70)
```

```

print("1. DATASET INFORMATION")
print("=" * 70)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nFirst 5 Records:")
display(df.head())

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())

# -----
# 4. REMOVE DUPLICATES
# -----
df = df.drop_duplicates()

print("\nAfter removing duplicates:")
print("Rows:", len(df))

# -----
# 5. IDENTIFY TIMESTAMP COLUMN
# -----
possible_time_columns = [
    'Timestamp',
    'timestamp',
    'Date',
    'date',
    'Datetime',
    'datetime'
]

time_column = None

for col in possible_time_columns:
    if col in df.columns:
        time_column = col
        break

if time_column is None:
    raise ValueError(
        "Timestamp/Date column not found. "
        "Please check the column name in your dataset."
    )

df[time_column] = pd.to_datetime(

```

```

    df[time_column],
    errors='coerce'
)

df = df.dropna(subset=[time_column])

# Rename timestamp column
df['Timestamp'] = df[time_column]

# -----
# 6. FIND POLLUTANT COLUMNS
# -----
pollutant_keywords = {
    'PM2.5': ['PM2.5', 'PM2_5', 'PM25'],
    'PM10': ['PM10'],
    'NO2': ['NO2'],
    'SO2': ['SO2'],
    'CO': ['CO'],
    'O3': ['Ozone', 'O3']
}

pollutant_columns = {}

for standard_name, possible_names in pollutant_keywords.items():

    for col in df.columns:

        col_upper = col.upper()

        for name in possible_names:

            if name.upper() in col_upper:
                pollutant_columns[standard_name] = col
                break

        if standard_name in pollutant_columns:
            break

print("\nDetected Pollutants:")

for standard_name, column in pollutant_columns.items():
    print(f"{standard_name} -> {column}")

# -----
# 7. METEOROLOGICAL VARIABLES
# -----
meteorological_keywords = {
    'Temperature': ['AT', 'Temperature', 'Temp'],
    'Humidity': ['RH', 'Humidity'],
    'Wind Speed': ['WS', 'Wind Speed', 'WindSpeed']
}

```

```

meteorological_columns = {}

for standard_name, possible_names in meteorological_keywords.items():

    for col in df.columns:

        col_upper = col.upper()

        for name in possible_names:

            if name.upper() in col_upper:
                meteorological_columns[standard_name] = col
                break

        if standard_name in meteorological_columns:
            break

print("\nDetected Meteorological Variables:")

for standard_name, column in meteorological_columns.items():
    print(f"{standard_name} -> {column}")

# -----
# 8. CONVERT NUMERIC VARIABLES
# -----
analysis_columns = list(
    pollutant_columns.values()
) + list(
    meteorological_columns.values()
)

for col in analysis_columns:

    df[col] = pd.to_numeric(
        df[col],
        errors='coerce'
    )

# -----
# 9. CREATE TIME FEATURES
# -----
df['Year'] = df['Timestamp'].dt.year
df['Month'] = df['Timestamp'].dt.month
df['Month_Name'] = df['Timestamp'].dt.month_name()
df['Day'] = df['Timestamp'].dt.day

def get_season(month):

    if month in [12, 1, 2]:
        return 'Winter'

```

```

    elif month in [3, 4, 5]:
        return 'Summer'

    elif month in [6, 7, 8, 9]:
        return 'Monsoon'

    else:
        return 'Post-Monsoon'

df['Season'] = df['Month'].apply(get_season)

print("\nTime features created successfully.")

# =====
# EXPLORATORY DATA ANALYSIS
# =====

# -----
# 10. DESCRIPTIVE STATISTICS
# -----
print("\n" + "=" * 70)
print("2. DESCRIPTIVE STATISTICS")
print("=" * 70)

display(
    df[analysis_columns].describe().T
)

# -----
# 11. AVERAGE POLLUTANT LEVELS
# -----
print("\nAverage Pollutant Concentrations:")

pollutant_means = {}

for standard_name, column in pollutant_columns.items():

    pollutant_means[standard_name] = df[column].mean()

mean_df = pd.DataFrame(
    pollutant_means.items(),
    columns=['Pollutant', 'Average']
)

display(mean_df)

# -----
# 12. BAR CHART - AVERAGE POLLUTION
# -----
plt.figure(figsize=(10, 6))

```

```

plt.bar(
    mean_df['Pollutant'],
    mean_df['Average']
)

plt.title(
    'Average Pollutant Concentration'
)

plt.xlabel('Pollutant')
plt.ylabel('Average Concentration')

plt.grid(
    axis='y',
    alpha=0.3
)

plt.show()

# =====
# MONTHLY / SEASONAL ANALYSIS
# =====

# -----
# 13. MONTHLY POLLUTION
# -----
monthly_data = df.groupby('Month')

monthly_pollution = pd.DataFrame()

for standard_name, column in pollutant_columns.items():

    monthly_pollution[standard_name] = (
        monthly_data[column].mean()
    )

print("\n" + "=" * 70)
print("3. MONTHLY POLLUTION ANALYSIS")
print("=" * 70)

display(monthly_pollution)

# -----
# 14. MONTHLY LINE GRAPH
# -----
plt.figure(figsize=(14, 7))

for pollutant in monthly_pollution.columns:

    plt.plot(

```

```

        monthly_pollution.index,
        monthly_pollution[pollutant],
        marker='o',
        label=pollutant
    )

plt.title(
    'Monthly Average Pollutant Concentrations'
)

plt.xlabel('Month')
plt.ylabel('Average Concentration')

plt.xticks(range(1, 13))

plt.legend()

plt.grid(
    alpha=0.3
)

plt.show()

# -----
# 15. SEASONAL ANALYSIS
# -----
season_order = [
    'Winter',
    'Summer',
    'Monsoon',
    'Post-Monsoon'
]

seasonal_pollution = df.groupby(
    'Season'
)[list(pollutant_columns.values())].mean()

seasonal_pollution = seasonal_pollution.reindex(
    season_order
)

print("\n" + "=" * 70)
print("4. SEASONAL POLLUTION ANALYSIS")
print("=" * 70)

display(seasonal_pollution)

# -----
# 16. SEASONAL PM2.5 BOX PLOT
# -----
if 'PM2.5' in pollutant_columns:

```



```

pm25_column = pollutant_columns['PM2.5']

plt.figure(figsize=(10, 6))

sns.boxplot(
    data=df,
    x='Season',
    y=pm25_column,
    order=season_order
)

plt.title(
    'Seasonal Distribution of PM2.5'
)

plt.xlabel('Season')
plt.ylabel('PM2.5 Concentration')

plt.show()

# =====
# CORRELATION ANALYSIS
# =====

# -----
# 17. CORRELATION MATRIX
# -----
correlation_columns = (
    list(pollutant_columns.values())
    +
    list(meteorological_columns.values())
)

correlation_columns = list(
    dict.fromkeys(correlation_columns)
)

correlation_matrix = df[
    correlation_columns
].corr()

print("\n" + "=" * 70)
print("5. CORRELATION ANALYSIS")
print("=" * 70)

display(correlation_matrix)

# -----
# 18. CORRELATION HEATMAP
# -----

```

```
plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title(
    'Correlation Between Pollutants and Meteorological Variables'
)

plt.show()
```

```
# =====
# SCATTER PLOTS
# =====
```

```
# -----
# 19. PM2.5 VS TEMPERATURE
# -----
if (
    'PM2.5' in pollutant_columns
    and
    'Temperature' in meteorological_columns
):

    pm25 = pollutant_columns['PM2.5']
    temp = meteorological_columns['Temperature']

    plt.figure(figsize=(8, 6))

    sns.scatterplot(
        data=df,
        x=temp,
        y=pm25
    )

    sns.regplot(
        data=df,
        x=temp,
        y=pm25,
        scatter=False
    )

    plt.title(
        'PM2.5 vs Temperature'
    )
```

```

plt.xlabel('Temperature')
plt.ylabel('PM2.5')

plt.show()

# -----
# 20. PM2.5 VS WIND SPEED
# -----
if (
    'PM2.5' in pollutant_columns
    and
    'Wind Speed' in meteorological_columns
):

    pm25 = pollutant_columns['PM2.5']
    wind = meteorological_columns['Wind Speed']

    plt.figure(figsize=(8, 6))

    sns.scatterplot(
        data=df,
        x=wind,
        y=pm25
    )

    sns.regplot(
        data=df,
        x=wind,
        y=pm25,
        scatter=False
    )

    plt.title(
        'PM2.5 vs Wind Speed'
    )

    plt.xlabel('Wind Speed')
    plt.ylabel('PM2.5')

    plt.show()

# =====
# ANOMALY DETECTION
# =====

# -----
# 21. IQR ANOMALY DETECTION
# -----
print("\n" + "=" * 70)
print("6. ANOMALY DETECTION")
print("=" * 70)

```

```

anomaly_results = {}

for pollutant_name, column in pollutant_columns.items():

    values = df[column].dropna()

    if len(values) == 0:
        continue

    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    anomalies = df[
        (df[column] < lower_limit)
        |
        (df[column] > upper_limit)
    ]

    anomaly_results[pollutant_name] = {
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower Limit': lower_limit,
        'Upper Limit': upper_limit,
        'Number of Anomalies': len(anomalies)
    }

anomaly_df = pd.DataFrame(
    anomaly_results
).T

display(anomaly_df)

# -----
# 22. PM2.5 ANOMALY GRAPH
# -----
if 'PM2.5' in pollutant_columns:

    pm25_column = pollutant_columns['PM2.5']

    values = df[pm25_column].dropna()

    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)

```

IQR = Q3 - Q1

upper_limit = Q3 + 1.5 * IQR

plt.figure(figsize=(14, 6))

**plt.plot(
 df['Timestamp'],
 df[pm25_column],
 linewidth=0.8
)**

**plt.axhline(
 upper_limit,
 linestyle='--',
 label='Anomaly Threshold'
)**

**plt.title(
 'PM2.5 Anomaly Detection'
)**

**plt.xlabel('Date')
plt.ylabel('PM2.5')**

plt.legend()

plt.grid(alpha=0.3)

plt.show()

```
# =====  
# DAILY POLLUTION ANALYSIS  
# =====
```

```
# -----  
# 23. HIGHEST POLLUTION DAYS  
# -----  
if 'PM2.5' in pollutant_columns:
```

pm25_column = pollutant_columns['PM2.5']

**daily_pm25 = (
 df.groupby(
 df['Timestamp'].dt.date
)[pm25_column]
 .mean()
 .sort_values(
 ascending=False
)
)**

```
print("\n" + "=" * 70)
print("7. TOP 10 HIGHEST PM2.5 DAYS")
print("=" * 70)
```

```
display(
    daily_pm25.head(10)
)
```

```
# -----
# 24. TOP 10 DAYS GRAPH
# -----
```

```
top_days = (
    daily_pm25
    .head(10)
    .sort_values()
)
```

```
plt.figure(figsize=(10, 6))
```

```
top_days.plot(
    kind='barh'
)
```

```
plt.title(
    'Top 10 Highest PM2.5 Days'
)
```

```
plt.xlabel(
    'Average PM2.5'
)
```

```
plt.ylabel(
    'Date'
)
```

```
plt.show()
```

```
# =====
# GEOGRAPHIC VISUALIZATION
# =====
```

```
# -----
# 25. MAP
# -----
```

```
print("\n" + "=" * 70)
print("8. GEOGRAPHIC VISUALIZATION")
print("=" * 70)
```

```
# Approximate Sirifort CPCB monitoring station
```

```
latitude = 28.5494
longitude = 77.2106
```

```
air_quality_map = folium.Map(
    location=[
        latitude,
        longitude
    ],
    zoom_start=11
)
```

```
folium.Marker(
    location=[
        latitude,
        longitude
    ],
    popup='CPCB Sirifort Air Quality Monitoring Station',
    tooltip='Sirifort CPCB Station'
).add_to(
    air_quality_map
)
```

```
display(air_quality_map)
```

```
# =====
# FINAL ANALYSIS & RECOMMENDATIONS
# =====
```

```
print("\n" + "=" * 70)
print("9. FINAL AIR QUALITY ASSESSMENT")
print("=" * 70)
```

```
# Highest average pollutant
```

```
highest_pollutant = (
    mean_df
    .sort_values(
        'Average',
        ascending=False
    )
    .iloc[0]
)
```

```
print(
    f"\nHighest average pollutant: "
    f"{highest_pollutant['Pollutant']}"
)
```

```
print(
    f"Average concentration: "
    f"{highest_pollutant['Average']:.2f}"
)
```

```
# Highest PM2.5 season
if 'PM2.5' in pollutant_columns:
```

```
    pm25_column = pollutant_columns['PM2.5']
```

```
    seasonal_pm25 = (
        df.groupby('Season')[pm25_column]
        .mean()
        .reindex(season_order)
    )
```

```
    highest_season = (
        seasonal_pm25.idxmax()
    )
```

```
    print(
        f"\nSeason with highest PM2.5: "
        f"{highest_season}"
    )
```

```
    print(
        f"Average PM2.5: "
        f"{seasonal_pm25.max():.2f}"
    )
```

```
    print(
        f"\nMaximum recorded PM2.5: "
        f"{df[pm25_column].max():.2f}"
    )
```

```
# Number of anomalies
if 'PM2.5' in anomaly_results:
```

```
    print(
        f"\nNumber of detected PM2.5 anomalies: "
        f"{anomaly_results['PM2.5']['Number of Anomalies']}"
    )
```

```
# Strongest correlations
print("\nStrongest correlations:")
```

```
corr_values = (
    correlation_matrix
    .where(
        np.triu(
            np.ones(
                correlation_matrix.shape
            ),
            k=1
        ).astype(bool)
    )
```



```

    )
    .stack()
    .sort_values(
        ascending=False
    )
)

print("\nPositive correlations:")
print(corr_values.head(5))

print("\nNegative correlations:")
print(corr_values.tail(5))

# =====
# DATA-DRIVEN RECOMMENDATIONS
# =====

print("\n" + "=" * 70)
print("10. DATA-DRIVEN ENVIRONMENTAL RECOMMENDATIONS")
print("=" * 70)

recommendations = [
    "1. Increase air-quality monitoring in areas showing persistent high pollution.",
    "2. Issue public health alerts during periods of extreme pollution.",
    "3. Promote public transportation during high-pollution periods.",
    "4. Investigate traffic and industrial emission sources in pollution hotspots.",
    "5. Use weather conditions such as wind speed and temperature to anticipate pollution events.",
    "6. Encourage pollution-control measures in industries and high-traffic areas.",
    "7. Increase public awareness about avoiding outdoor activities during severe pollution events.",
    "8. Continue long-term monitoring to identify seasonal and yearly pollution trends."
]

for recommendation in recommendations:
    print(recommendation)

# =====
# SAVE CLEANED DATASET
# =====

# -----
# 26. EXPORT CLEANED DATA
# -----
output_file = 'cleaned_air_quality_analysis.csv'

df.to_csv(
    output_file,
    index=False
)

print("\n" + "=" * 70)
print("PROJECT COMPLETED SUCCESSFULLY!")

```

```
print(
    f"\nCleaned dataset saved as: {output_file}"
)
```

```
files.download(
    output_file
)
```

OUTPUT

... Cleaned shape: (70176, 29)

	Station ID	State	City	Station Name	Timestamp	PM2.5 (µg/m³)	PM10 (µg/m³)	NO (µg/m³)	NO2 (µg/m³)	NOx (ppb)	...	MP-Xylene (µg/m³)
0	site_119	Delhi	Delhi	Sirifort, Delhi - CPCB	2024-01-01 00:00:00+00:00	199.10	290.0	22.79	81.70	61.99	...	0.17
1	site_119	Delhi	Delhi	Sirifort, Delhi - CPCB	2024-01-01 00:15:00+00:00	199.10	290.0	22.90	80.97	61.69	...	0.17
2	site_119	Delhi	Delhi	Sirifort, Delhi - CPCB	2024-01-01 00:30:00+00:00	199.10	290.0	21.92	78.59	59.62	...	0.16
3	site_119	Delhi	Delhi	Sirifort, Delhi - CPCB	2024-01-01 00:45:00+00:00	206.04	294.0	22.26	77.94	59.55	...	0.16
4	site_119	Delhi	Delhi	Sirifort, Delhi - CPCB	2024-01-01 01:00:00+00:00	227.20	305.0	21.85	77.82	59.16	...	0.15

5 rows × 29 columns

✓ 0s

```
df['Season'] = df['Month'].apply(get_season)

df[['Timestamp', 'Month_Name', 'Season']].head()
```



...

	Timestamp	Month_Name	Season
--	-----------	------------	--------



0	2024-01-01 00:00:00+00:00	January	Winter
1	2024-01-01 00:15:00+00:00	January	Winter
2	2024-01-01 00:30:00+00:00	January	Winter
3	2024-01-01 00:45:00+00:00	January	Winter
4	2024-01-01 01:00:00+00:00	January	Winter